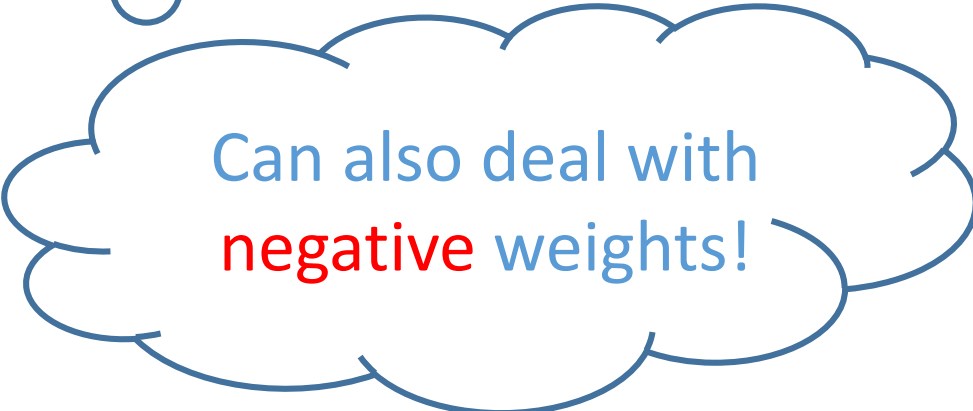


SSSP- Single Source Shortest Paths

Belman-Ford's Algorithm



Can also deal with
negative weights!



At extra cost...

SSSP- Single Source Shortest Paths

Belman-Ford's Algorithm Cost

Command Number	Cost
1	$\Theta(V)$
2-4	$\Theta(V \cdot E)$
5-7	$O(E)$
8	$\Theta(1)$
Total	$\Theta(V \cdot E)$

SSSP- Single Source Shortest Paths

Belman-Ford's Algorithm Correctness

Claim:

Let $G = (V, E)$ be a weighted digraph with $w: E \rightarrow \mathbb{R}$ and a source s .

After the **i-th** iteration of the for-loop in line 2 of Belman-Ford's algorithm, it holds that:

for every $v \in V$, $d[v]$ is the weight of a shortest path from s to v that uses at most **i-1** intermediate vertices.

SSSP- Single Source Shortest Paths

Belman-Ford's Algorithm Correctness

Proof (of claim): By induction on i .

Base Case: $i=1$

In this case $i-1=1-1=0$.

Therefore, no intermediate vertices can be used in the first iteration.

Indeed, by performing RELAX on all edges, the first iteration improves $d[v]$ values only for vertices v that are neighbors of s , having direct edge from s to v .

SSSP- Single Source Shortest Paths

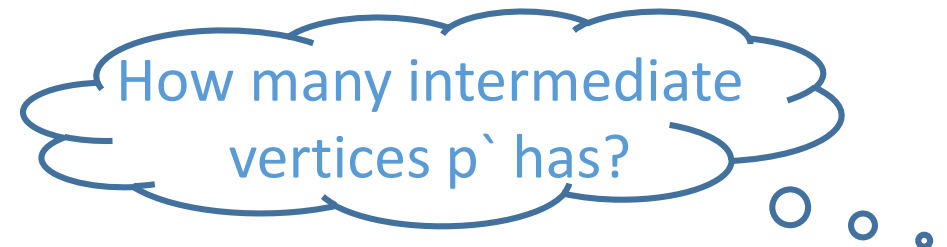
Belman-Ford's Algorithm Correctness

Proof (cont.):

Induction step:

Assume by induction claim holds for $1 \leq i < |V| - 1$. We prove the claim for iteration $i+1$.

Let $v \in V$ be a vertex for which there exists a shortest path from s to v that uses i intermediate vertices. Let u be the last vertex on this path before v .



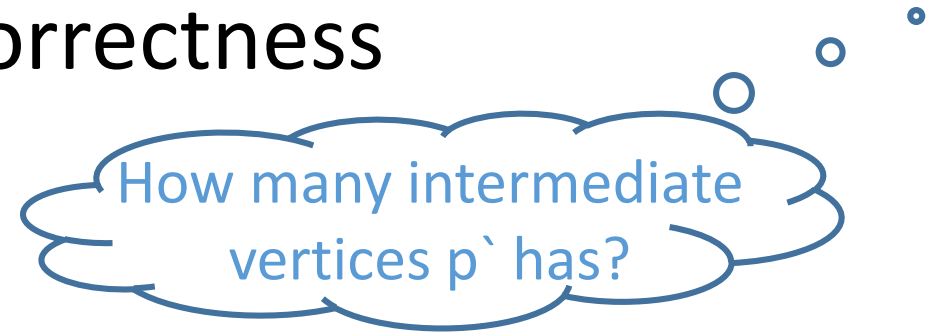
SSSP- Single Source Shortest Paths

Belman-Ford's Algorithm Correctness

Proof (cont.):



Since p' has $i-1$ intermediate vertices, and it is a shortest such path from s to u (why?), we have by induction hypothesis that $d[u]$ contains $w(p')$ after the i -th iteration.



SSSP- Single Source Shortest Paths

Belman-Ford's Algorithm Correctness

Proof (cont.):

$s \rightsquigarrow_{p'} u \rightarrow v$

Since p' has $i-1$ intermediate vertices, and it is a shortest such path from s to u (why?), we have by induction hypothesis that $d[u]$ contains $w(p')$ after the i -th iteration.

How many intermediate vertices p' has?

By sub-paths optimality of shortest paths

SSSP- Single Source Shortest Paths

Belman-Ford's Algorithm Correctness

Proof (cont.):



Since p' has $i-1$ intermediate vertices, and it is a shortest such path from s to u (why?),

we have by induction hypothesis that $d[u]$ contains $w(p')$ after the i -th iteration.

Since on the $i+1$ iteration a RELAX call was performed on every edge, including the call $\text{RELAX}(u,v,w)$, the claim then follows from property 3 of the RELAX procedure.

How many intermediate vertices p' has?

By sub-paths optimality of shortest paths

SSSP- Single Source Shortest Paths

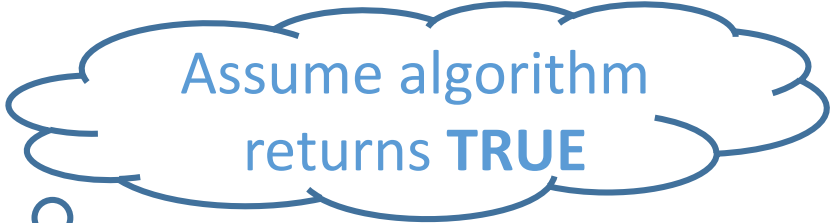
Belman-Ford's Algorithm Correctness

Conclusion:

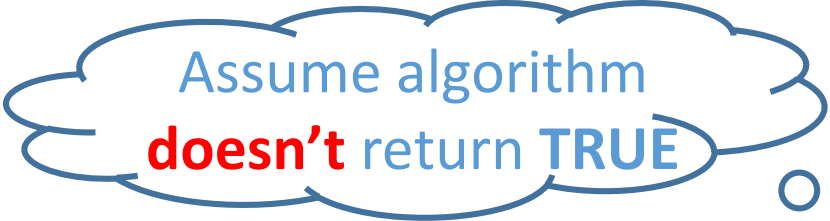
Let $G = (V, E)$ be a weighted digraph with $w: E \rightarrow \mathbb{R}$.
When Belman-Ford's algorithm halts, for every $v \in V$,
 $d[v] = \delta(s, v)$ if and only if the algorithm returns TRUE.

Proof (of conclusion):

We need to show both directions.



Assume algorithm
returns **TRUE**



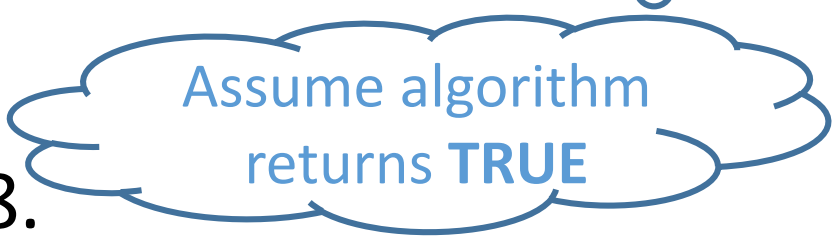
Assume algorithm
doesn't return **TRUE**

SSSP- Single Source Shortest Paths

Belman-Ford's Algorithm Correctness

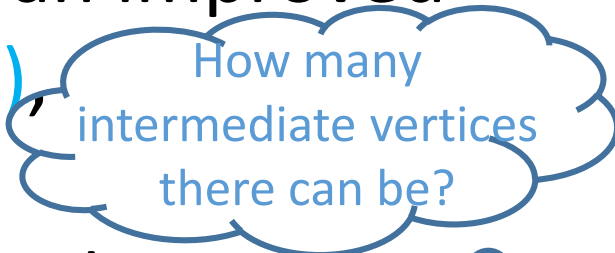
Proof (of conclusion):

Assume the algorithm returns TRUE in line 8.



Assume algorithm
returns **TRUE**

Then, each of the checks in lines 5-7 did not detect an improved path. Since these paths are necessarily cyclic (**why?**),



How many
intermediate vertices
there can be?

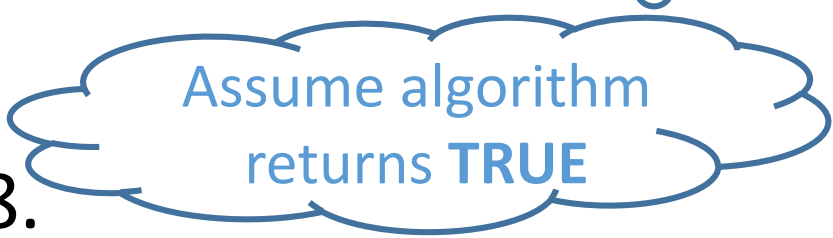
we conclude that there is no cycle with negative weight in G .

SSSP- Single Source Shortest Paths

Belman-Ford's Algorithm Correctness

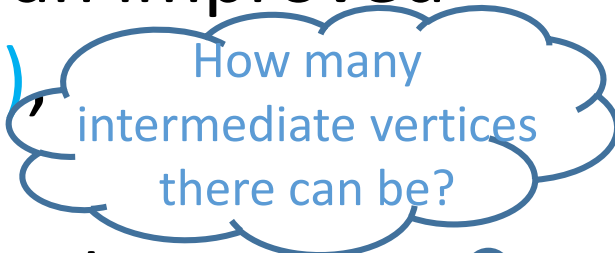
Proof (of conclusion):

Assume the algorithm returns TRUE in line 8.



Assume algorithm
returns **TRUE**

Then, each of the checks in lines 5-7 did not detect an improved path. Since these paths are necessarily cyclic (**why?**),



How many
intermediate vertices
there can be?

we conclude that there is no cycle with negative weight in G .

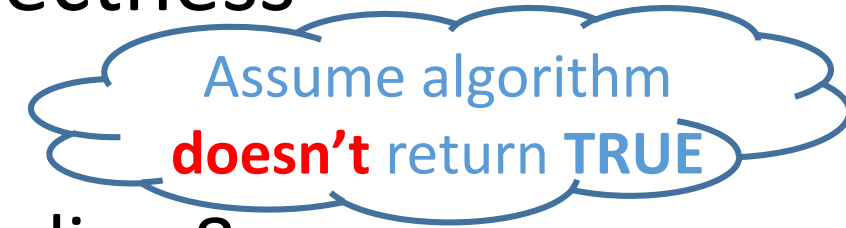
Thus, all the shortest paths in G are simple paths and, therefore, were detected and their weights correctly computed during the for-loop in lines 2-4, by **Claim**.

SSSP- Single Source Shortest Paths

Belman-Ford's Algorithm Correctness

Proof (of conclusion):

Assume the algorithm doesn't return TRUE in line 8.



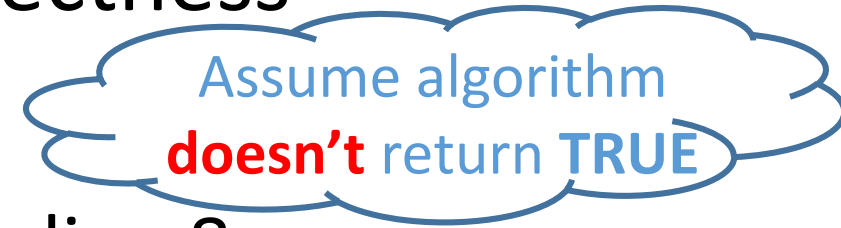
Then, the algorithm returned FALSE during one of the checks in lines 5-7.

SSSP- Single Source Shortest Paths

Belman-Ford's Algorithm Correctness

Proof (of conclusion):

Assume the algorithm doesn't return TRUE in line 8.



Then, the algorithm returned FALSE during one of the checks in lines 5-7.

This means that an improved path was detected. Since such a path is necessarily cyclic (**why?**), we conclude that there exists a cycle with negative weight in G .

In this case, for every vertex v on a path from s containing this cycle $\delta(s, v) = -\infty$, and is not defined.

Single Source Shortest Paths & Solving Differences Constraints

Definition: **Linear Programming**

Let A be an $m \times n$ matrix, b a length m vector and c a length n vector.

We wish to find a length n vector x , such that the target function:

$$\sum_{i=1}^n c_i \cdot x_i$$

has maximum value, subject to the m constraints given by:

$$A \cdot x \leq b.$$

Single Source Shortest Paths & Solving Differences Constraints

Focus on a special case of
linear programming

Definition: **Linear Programming**

Let A be an $m \times n$ matrix, b a length m vector and c length n vector.

We wish to find a length n vector x , such that the target function:

$$\sum_{i=1}^n c_i \cdot x_i$$

has maximum value, subject to the m constraints given by:

$$A \cdot x \leq b.$$

Single Source Shortest Paths & Solving Differences Constraints

Definition: **Differences Constraints**

Focus on a special case of
linear programming

Let A be an $m \times n$ matrix, b a length m vector.

We wish to find a length n vector x , subject to the m constraints given by:

$$A \cdot x \leq b,$$

where every row in A has only one 1 value and one -1 value, and every other value in A is 0.

We solve this problem
using SSSP solution

This technique is called
REDUCTION

Single Source Shortest Paths & Solving Differences Constraints

The Reduction from Differences Constraints to SSSP

Let A be an $m \times n$ matrix, b a length m vector both defining the constraints set C . We build the constraints-graph $G = (V, E)$ with weight function w as follows:

$$V = \{v_0, v_1, \dots, v_n\}$$
$$E = \{(v_0, v_i) | 1 \leq i \leq n\} \cup \{(v_i, v_j) \mid x_j - x_i \leq b_k \in C\}$$
$$w(v_i, v_j) = \begin{cases} 0, & \text{if } i = 0 \\ b_k, & \text{if } x_j - x_i \leq b_k \in C \end{cases}$$

Single Source Shortest Paths & Solving Differences Constraints

The Reduction from Differences Constraints to SSSP

We run Belman-Ford algorithm on the constructed constraints-graph $G = (V, E)$. If the algorithm returns TRUE, we return the vector: $X = \{d[v_1], \dots, d[v_n]\}$ as solution.

If the algorithms returns FALSE, we answer that there is no vector subject to the constraints.

Single Source Shortest Paths & Solving Differences Constraints

The Reduction from Differences Constraints to SSSP Correctness

We show that the all constraints hold for the returned solution vector X . Let $x_j - x_i \leq b_k$ be a constraint in C . Then, we get:

$x_j = d[v_j]$ and $x_i = d[v_i]$. We have that:

$$\delta(s, v_j) \leq \delta(s, v_i) + w(v_i, v_j)$$



By shortest paths
weight definition

Single Source Shortest Paths & Solving Differences Constraints

The Reduction from Differences Constraints to SSSP Correctness

We show that the all constraints hold for the returned solution vector X . Let $x_j - x_i \leq b_k$ be a constraint in C . Then, we get:

$x_j = d[v_j]$ and $x_i = d[v_i]$. We have that:

$$\delta(s, v_j) \leq \delta(s, v_i) + w(v_i, v_j)$$

$$\delta(s, v_j) - \delta(s, v_i) \leq w(v_i, v_j)$$



By shortest paths
weight definition

Single Source Shortest Paths & Solving Differences Constraints

The Reduction from Differences Constraints to SSSP Correctness

We show that the all constraints hold for the returned solution vector X . Let $x_j - x_i \leq b_k$ be a constraint in C . Then, we get:

$x_j = d[v_j]$ and $x_i = d[v_i]$. We have that:

$$\delta(s, v_j) \leq \delta(s, v_i) + w(v_i, v_j)$$

$$\delta(s, v_j) - \delta(s, v_i) \leq w(v_i, v_j)$$

$$d[v_j] - d[v_i] \leq w(v_i, v_j)$$

By shortest paths
weight definition

By Belman-Ford's
correctness

Single Source Shortest Paths & Solving Differences Constraints

The Reduction from Differences Constraints to SSSP Correctness

We show that the all constraints hold for the returned solution vector X . Let $x_j - x_i \leq b_k$ be a constraint in C . Then, we get:

$x_j = d[v_j]$ and $x_i = d[v_i]$. We have that:

$$\delta(s, v_j) \leq \delta(s, v_i) + w(v_i, v_j)$$

$$\delta(s, v_j) - \delta(s, v_i) \leq w(v_i, v_j)$$

$$d[v_j] - d[v_i] \leq w(v_i, v_j)$$

$$x_j - x_i \leq b_k$$

By solution
construction

By shortest paths
weight definition

By Belman-Ford's
correctness

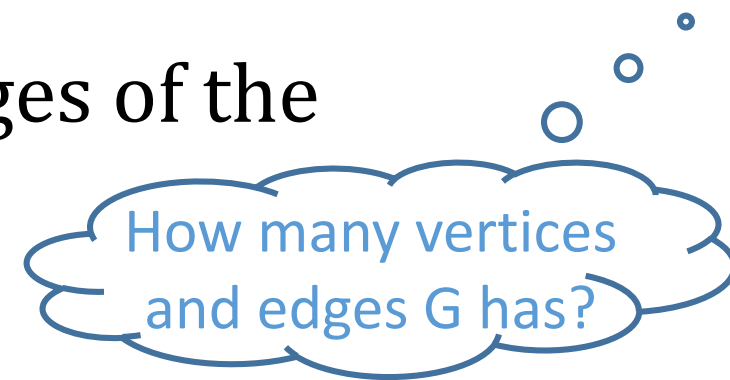
Single Source Shortest Paths & Solving Differences Constraints

The Reduction from Differences Constraints to SSSP Cost

Belman-Ford's algorithm costs $\Theta(|V| \cdot |E|)$.

By plugging in the number of vertices and edges of the constraints graph G , we get:

$$\Theta((n + 1) \cdot (m + n))$$



Note that G 's construction cost is less than that.